Разработка модулей ядра Linux: Часть 7. Анализ выполнения системного вызова

Олег Цилюрик
преподаватель тренингового отделения
Global Logic

15.05.2012

Эта статья продолжает цикл статей, посвященный написанию модулей ядра Linux. В статье подробно анализируется процесс выполнения системного вызова.

Введение

Продолжая изучение отличий между модулем ядра и пользовательским процессом, вернемся к технике осуществления системного вызова из процесса. Рассмотрение системных вызовов, приведенное в прошлых частях цикла, было исключительно теоретическим, и сейчас можно это компенсировать. Кроме того, именно представленный пример лучше всего продемонстрирует нюансы различий, описанные выше.

Пример системного вызова

Все обсуждаемые примеры содержатся в архиве int80.tgz (который можно найти в разделе Материалы для скачивания), и, в отличие от всех остальных примеров, они применимы только к архитектуре Intel x86 , так как в них реализуется прямой системный вызов Linux через команду ассемблера **int 80h**. Первый пример (файл **mp.c**) демонстрирует пользовательский процесс, последовательно выполняющий системные вызовы, эквивалентные библиотечным: **getpid(), write(), mknod()**, причём **write()** выполняется именно на дескриптор 1, то есть **printf()**.

Листинг 1. Обычный пользовательский процесс

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <sys/stat.h>
#include <linux/kdev_t.h>
#include <sys/syscall.h>

int write_call( int fd, const char* str, int len ) {
   long __res;
   __asm__ volatile ( "int $0x80":
      "=a" (__res):"0"(__NR_write),"b"((long)(fd)),"c"((long)(str)),"d"((long)(len)) );
   return (int) __res;
}

void do_write( void ) {
```

© Copyright IBM Corporation 2012

```
   char *str = "эталонная строка для вывода!\n";
   int len = strlen( str ) + 1, n;
   printf( "string for write length = %d\n", len );
   n = write_call( 1, str, len );
   printf( "write return : %d\n", n );
}

int mknod_call( const char *pathname, mode_t mode, dev_t dev ) {
   long __res;
   __asm__ volatile ( "int $0x80":
      "=a" (__res):
      "a"(__NR_mknod),"b"((long)(pathname)),"c"((long)(mode)),"d"((long)(dev))
   );
   return (int) __res;
};

int mknod_call( const char *pathname, mode_t mode, dev_t dev ) {
   long __res;
   __asm__ volatile ( "int $0x80":
      "=a" (__res):
      "a"(__NR_mknod),"b"((long)(pathname)),"c"((long)(mode)),"d"((long)(dev))
   );
   return (int) __res;
};

void do_mknod( void ) {
   char *nam = "ZZZ";
   int n = mknod_call( nam, S_IFCHR | S_IRUSR | S_IWUSR, MKDEV( 247, 0 ) );
   printf( "mknod return : %d\n", n );
}

int getpid_call( void ) {
   long __res;
   __asm__ volatile ( "int $0x80":"=a" (__res):"a"(__NR_getpid) );
   return (int) __res;
};

void do_getpid( void ) {
   int n = getpid_call();
   printf( "getpid return : %d\n", n );
}

int main( int argc, char *argv[] ) {
   do_getpid();
   do_write();
   do_mknod();
   return EXIT_SUCCESS;
};
```

Пример написан с использованием ассемблерных inline-вставок компилятора GCC, которые будут рассмотрены в отдельной статье. Пример прост и интуитивно понятен: в каждом случае регистры загружаются значениями из переменных С-кода и вызывается прерывание. Ниже приведен результат его запуска:

```
$ ./mp
getpid return : 14180
string for write length = 54
эталонная строка для вывода!
write return : 54
mknod return : -1
```

Всё хорошо, за исключением вызова **mknod()**, но стоит вспомнить, что одноимённая консольная команда требует прав root:

```
$ sudo ./mp
getpid return : 14182
string for write length = 54
эталонная строка для вывода!
write return : 54
mknod return : 0
$ ls -l ZZZ
crw------- 1 root root 247, 0 Дек 20 22:00 ZZZ
$ rm ZZZ
rm: удалить защищенный от записи знаковый специальный файл `ZZZ'? y
```

В результате запуска программы удалось создать именованное символьное устройство, при этом не в каталоге **/dev** (где оно должно находиться), а в текущем рабочем каталоге. Но желательно потом удалить это имя, как показано в последней команде.

Рассмотрим следующий вопрос: нельзя ли выполнить эти (а значит и другие) системные вызовы из кода модуля ядра, то есть изнутри ядра? Оформим код, фактически совпадающий с приведенным в листинге 1, в форме модуля ядра. Но так как хотелось бы написать два почти идентичных модуля (**mdu.c** и **mdc.c**), то код, общий для них, необходимо поместить в общий включаемый файл (**syscall.h**):

Листинг 2. Код, общий для двух модулей

```
#include <linux/module.h>
#include <linux/fs.h>
#include <linux/sched.h>

int write_call( int fd, const char* str, int len ) {
   long __res;
   __asm__ volatile ( "int $0x80":
      "=a" (__res):"0"(__NR_write),"b"((long)(fd)),"c"((long)(str)),"d"((long)(len)) );
   return (int) __res;
}

void do_write( void ) {
   char *str = "=== эталонная строка для вывода!\n";
   int len = strlen( str ) + 1, n;
   printk( "=== string for write length = %d\n", len );
   n = write_call( 1, str, len );
   printk( "=== write return : %d\n", n );
}

int mknod_call( const char *pathname, mode_t mode, dev_t dev ) {
   long __res;
   __asm__ volatile ( "int $0x80":
      "=a" (__res):
      "a"(__NR_mknod),"b"((long)(pathname)),"c"((long)(mode)),"d"((long)(dev))
   );
   return (int) __res;
};

void do_mknod( void ) {
   char *nam = "ZZZ";
   int n = mknod_call( nam, S_IFCHR | S_IRUGO, MKDEV( 247, 0 ) );
   printk( KERN_INFO "=== mknod return : %d\n", n );
}

int getpid_call( void ) {
   long __res;
   __asm__ volatile ( "int $0x80":"=a" (__res):"a"(__NR_getpid) );
   return (int) __res;
};
```

```
void do_getpid( void ) {
   int n = getpid_call();
   printk( "=== getpid return : %d\n", n );
}
```

В листинге 3 приведен первый модуль **mdu.c**, который практически полностью повторяет код выполнявшегося выше процесса.

Листинг 3. Код модуля, выполненного по правилам

```
#include "syscall.h"

static int __init x80_init( void ) {
   do_write();
   do_mknod();
   do_getpid();
   return -1;
}

module_init( x80_init );
```

Результат его запуска показан ниже:

```
$ sudo insmod mdu.ko
insmod: error inserting 'mdu.ko': -1 Operation not permitted
$ dmesg | tail -n30 | grep ===
=== string for write length = 58
=== write return : -14
=== mknod return : -14
=== getpid return : 14217
$ ps -A | grep 14217
$
```

В общем, всё совершенно ожидаемо (ошибки выполнения), кроме вызова **getpid()**, который вызывает некоторые подозрения, но об этом позже. Цель успешно достигнута: было показано главное различие между пользовательским процессом и ядром. Оно состоит в том, что при выполнении системного вызова из любого процесса, код обработчика системного вызова (в ядре!) должен копировать **данные параметров** вызова из адресного пространства процесса в пространство ядра, а после выполнения копировать **данные результатов** обратно в адресное пространства процесса. А при попытке вызвать системный обработчик из контекста ядра (модуля), что только что было сделано, адресного пространства процесса не существует, так как нет самого процесса! Однако **getpid()** успешно выполнился и показал PID какого-то процесса. Он выполнился, так как этот системный вызов не получает параметров и не копирует результатов (он возвращает значение в регистре). А возвращен был PID того процесса, в **контексте** которого выполнялся системный вызов, т.е. процесса **insmod**. Но всё-таки системный вызов был выполнен из модуля!

Далее необходимо переписать модуль mdc.c, незначительно изменив пример из листинга 2.

Листинг 3. Код модуля, выполненного в нарушение правил

```
#include <linux/uaccess.h>
#include "syscall.h"

static int __init x80_init( void ) {
   mm_segment_t fs = get_fs();
   set_fs( get_ds() );
   do_write();
   do_mknod();
   do_getpid();
   set_fs( fs );
   return -1;
}

module_init( x80_init );
```

Примечание. Вызовы **set_fs()**, **get_ds()** выполняют смену сегмента данных с пользователя на ядро, но они будут рассматриваться в следующих статьях.

Результат запуска модуля **mdc.c**, приведенный ниже, может сильно удивить.

```
$ sudo insmod mdc.ko
=== эталонная строка для вывода!
insmod: error inserting 'mdc.ko': -1 Operation not permitted
$ dmesg | tail -n30 | grep ===
=== string for write length = 58
=== write return : 58
=== mknod return : 0
=== getpid return : 14248
$ ls -l ZZZ
cr--r--r-- 1 root root 0, 63232 Дек 20 22:04 ZZZ
```

Ранее говорилось, что модуль ядра не сможет выполнить **printf()** и осуществить вывод на графический терминал. Однако видно, что перед инсталляционным сообщением была выведена текстовая строка! На какой управляющий терминал был тогда произведен вывод? Конечно, на терминал запускающей программы **insmod**, и сделать подобное можно **только из функции инициализации** модуля. Но главное не это, а то, что в этом примере все системные вызовы **успешно** выполнились! А значит, выполнится и любой системный вызов, предназначенный для пользовательского пространства.

Заключение

Подобные эксперименты помогают достичь ясности в понимании взаимосвязей между процессами, ядром и модулями. Поэтому рекомендуется тщательно изучить представленные примеры или создать свои, а следующая статья будет посвящена интерфейсам, используемым модулями ядра.

Загрузка

Описание	Имя	Размер
	int80.tgz	2KB

Об авторе

Олег Цилюрик



Олег Иванович Цилюрик, много лет был разработчиком программного обеспечения в крупных центрах разработки: ВНИИ РТ, НПО "Дельта", КБ ПМ. Последние годы работал над проектами в области промышленной автоматики, IP телефонии и коммуникаций. Автор нескольких книг. Преподаватель тренингового отделения международной софтверной компании Global Logic.

© Copyright IBM Corporation 2012
(www.ibm.com/legal/copytrade.shtml)
Торговые марки
(www.ibm.com/developerworks/ru/ibm/trademarks/)